QML Torch Activity One Deep Dive Notes

1. Core Concept

- Feature maps transform raw inputs into richer spaces.
- Logistic regression is a linear model that works well on straight-line separable data but fails on curved structures.
- By adding nonlinear features (sine, cosine, polynomial terms), the same model can learn curved boundaries.
- This mirrors what quantum circuits do: map data into high-dimensional Hilbert space, making patterns separable.

Key Insight:
A model's performance depends not only on its architecture but on the features it is given.

---

2. Step-by-Step Walkthrough

A. Dataset and Utilities (_util.py)

- Library: sklearn.datasets.make_moons
    - n_samples=300: total points.
    - noise=0.2: introduces randomness.
    - random_state=42: ensures reproducibility.
- Split: train_test_split divides into training (80%) and testing (20%).

Conceptual Note:
Training data functions as practice (flashcards). Testing data evaluates generalization (the quiz).

Further Reading:

- https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html
- https://towardsdatascience.com/train-test-split-and-cross-validation-in-python-80b61beca4b6

B. Visualization (plotting.py)

- Libraries: numpy, matplotlib.pyplot
- Functions:
    - plot_dataset(X, y): produces a scatter plot of data.
    - plot_decision_boundary(model, X, y, transform=None): overlays decision boundary of trained model.

Technical Details:

- np.meshgrid: creates a grid of evaluation points.
- contourf: colors classification regions.
- scatter: plots original dataset points.

Further Reading:

- https://matplotlib.org/stable/gallery/images_contours_and_fields/contour_demo.html
- https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html

---

C. Feature Engineering (feature_maps.py)

- Classical Map: returns only x_1 and x_2.
- Qinspired Map:
    - Original features: x_1, x_2

    - Added features: $\sin(x_1)$, $\cos(x_2)$, $x_1 \cdot x_2$, x_1^2, x_2^2
- Custom Map: allows experimentation with new transformations.

Key Idea:
New features provide the model with richer perspectives, enabling curved decision boundaries.

Further Reading:

- https://towardsdatascience.com/feature-engineering-for-machine-learning-3a5e293a5114
- https://qiskit.org/textbook/ch-machine-learning/quantum-feature-maps.html

---

D. Main Runner (run_playground.py)

- Imports: argparse, numpy, Pipeline, StandardScaler, LogisticRegression
- CLI Argument: --mode classical | qinspired | custom
- Pipeline:
    1. StandardScaler → normalizes features.
    2. Feature Map → applies chosen transformation.
    3. LogisticRegression → trains linear model.
- Evaluation: pipeline.score(X_test, y_test) reports accuracy.
- Visualization: plot_decision_boundary displays separator.

Results:

- Classical Mode: Accuracy ~80%, straight-line boundary.
- Qinspired Mode: Accuracy ~95%, curved boundary.

Further Reading:

- https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
- https://cs229.stanford.edu/notes2022fall/notes2022fall/cs229-notes1.pdf

---

3. Conceptual Bridge to Quantum ML

- Quantum circuits act as feature maps in Hilbert space.
- These transformations are nonlinear, expressive, and difficult to simulate classically.
- Logistic regression or other simple models can achieve high performance once data is expressed in this richer space.

Further Reading:

- https://pennylane.ai/qml/demos/tutorial_kernel_intro.html
- https://arxiv.org/abs/1803.07128

---

4. Exploration Exercises

- Increase dataset noise to test robustness.
- Add new transformations, e.g., exp(x1) or log(|x2|+1).
- Replace logistic regression with SVM and compare kernels.
- Explore differences between manually engineered features and automatically learned features (neural networks).

---

5. Key Takeaways

1. Logistic regression struggles on non-linear data.

2. Adding feature maps transforms the input space, enabling curved decision boundaries.

3. The mechanism mirrors quantum feature maps, where quantum circuits generate rich transformations.

4. The improvement comes not from changing the model but from changing the representation of the data.